

UNIALFA - UNIVERSIDADE ALVES FARIAS
ENGENHARIA DE SOFTWARE
ANÁLISE E DESENVOLVIMENTO DE SISTEMAS

ARTHUR BRUNO CESAR SILVA
GUILHERME NUNES DA CRUZ
ROMULO GOMES CAVALCANTE

NURA: DESENVOLVIMENTO DE UMA PLATAFORMA SaaS
MULTITENANT DE GESTAO EDUCACIONAL

Goiânia

2026

ARTHUR BRUNO CESAR SILVA
GUILHERME NUNES DA CRUZ
ROMULO GOMES CAVALCANTE

NURA: DESENVOLVIMENTO DE UMA PLATAFORMA SaaS
MULTITENANT DE GESTAO EDUCACIONAL

Trabalho de Conclusão de Curso apresentado como requisito parcial para obtenção do título de Bacharel ou Tecnólogo, pelo Curso de Engenharia de Software e Sistemas de Informações da Universidade Alves Farias.

Orientador: Prof. Ricardo André Naka
Coorientador: Marcos Dias de Paula

Goiânia

2026

ARTHUR BRUNO CESAR SILVA, GUILHERME NUNES DA CRUZ, ROMULO GOMES CAVALCANTE, Nura: Desenvolvimento de Uma Plataforma SaaS Multi-Tenant de Gestão Educacional. Trabalho de Graduação Integrado (TGI) apresentado a Faculdade de Engenharia de Software e Sistemas da Informação, da Universidade Alves Farias para obtenção do título de Bacharel em Engenharia de Software e Sistemas de Informação.

Aprovado em: ____/____/____

Banca Examinadora

Prof. Dr. _____ - Instituição

Prof. Dr. _____ - Instituição

Prof. Dr. _____ - Instituição

Dedico este trabalho as nossas famílias, que sempre acreditaram no nosso potencial e nos apoiaram em cada etapa desta jornada acadêmica e profissional. Dedico também a todos os profissionais de tecnologia que, como nos, acreditam que o software pode transformar instituições de ensino e melhorar a vida de estudantes, professores e colaboradores.

AGRADECIMENTOS

Agradecemos primeiramente a Deus, por nos conceder saúde, determinação e clareza ao longo de toda a graduação.

A UNIALFA Universidade Alves Farias, pelo ambiente acadêmico que proporcionou crescimento intelectual e profissional durante estes anos de formação.

Ao Prof. Ricardo André Naka, nosso orientador, pela disponibilidade, pelas orientações precisas e pelo incentivo constante ao longo do desenvolvimento deste trabalho.

Ao Prof. Marcos Dias de Paula, nosso coorientador, pelo acompanhamento técnico e pelas contribuições que enriqueceram o desenvolvimento deste trabalho.

Aos professores do curso de Engenharia de Software e Sistemas de Informações, que contribuíram diretamente para nossa formação técnica e crítica.

Ao parceiro de desenvolvimento Romulo, cuja colaboração foi fundamental para a construção do sistema NURA, juntamente com Guilherme Nunes atuando periodicamente na construção e formalização de documentos, tornando este projeto possível com qualidade e profissionalismo.

As nossas famílias, pelo apoio incondicional, pela paciência nos momentos de dificuldade e pelo incentivo em cada conquista.

*"A sabedoria suprema é ter sonhos bastante
grandes para não se perderem de vista
enquanto os perseguimos."*

William Faulkner

*"Qualquer tolo pode escrever código que um
computador entende. Bons programadores
escrevem código que humanos entendem."*

Martin Fowler

RESUMO

SILVA, Arthur Bruno Cesar; CRUZ, Guilherme Nunes da; CAVALCANTE, Romulo Gomes. Nura: Desenvolvimento de Uma Plataforma SaaS Multi-Tenant de Gestao Educacional. 2026. Trabalho de Conclusão de Curso (Bacharelado em Engenharia de Software e Sistemas de Informação) - Universidade Alves Farias, Goiania, 2026.

As instituições de ensino superior brasileiras enfrentam crescentes desafios na gestão integrada de seus processos acadêmicos, administrativos e financeiros. A fragmentação de sistemas e o uso de planilhas como ferramenta de controle resultam em retrabalho, inconsistência de dados e dificuldades na tomada de decisão estratégica. O presente trabalho descreve o desenvolvimento do NURA, uma plataforma SaaS (Software as a Service) multi-tenant de gestão educacional construída com tecnologias modernas e open source. O sistema foi projetado para atender múltiplas instituições de ensino em uma única instancia, garantindo isolamento completo de dados entre organizações por meio da estrategia de row-level isolation com o campo organizacaoId. A arquitetura do NURA e baseada em um monorepo que separa o backend desenvolvido em NestJS com TypeScript e Prisma ORM sobre PostgreSQL do frontend, construído em React 18 com Redux Toolkit e Tailwind CSS v4. O sistema implementa controle de acesso baseado em papeis RBAC com 15 perfis de usuário distintos, cobrindo desde o Super Administrador da plataforma até estudantes, candidatos ao processo seletivo e egressos. Entre os módulos desenvolvidos destacam-se: gestão acadêmica (cursos, turmas, disciplinas, matrículas, notas e frequência), modulo financeiro (mensalidades, cobranças e pagamentos), secretaria acadêmica, comunicação, trabalhos de conclusão de curso e estagio. A qualidade do código e garantida por TypeScript em modo estrito, ESLint, pre-commit hooks com Husky e testes automatizados com Jest e Cypress. Os resultados demonstram a viabilidade técnica e econômica da solução, que opera em ambiente de produção atendendo instituições reais, validando sua aplicabilidade além do contexto acadêmico.

Palavras-chave: gestão educacional; SaaS; multi-tenant; NestJS; React; TypeScript; RBAC; engenharia de software.

ABSTRACT

SILVA, Arthur Bruno Cesar; CRUZ, Guilherme Nunes da; CAVALCANTE, Romulo Gomes. Nura: Development of a Multi-Tenant SaaS Platform for Educational Management. 2026. Undergraduate Thesis (Bachelor in Software Engineering and Information Systems) - Universidade Alves Farias, Goiania, 2026.

Brazilian higher education institutions face growing challenges in the management of their academic, administrative, and financial processes. The fragmentation of systems and the use of spreadsheets as control tools result in rework, data inconsistency, and difficulties in strategic decision-making. This work describes the development of NURA, a multi-tenant educational management SaaS (Software as a Service) platform built with modern open-source technologies. The system was designed to serve multiple educational institutions within a single instance, ensuring complete data isolation between organizations through a row-level isolation strategy using the `organizacaoId` field. The NURA architecture is based on a monorepo that separates the backend, developed with NestJS, TypeScript, and Prisma ORM on PostgreSQL, from the frontend, built with React 18, Redux Toolkit, and Tailwind CSS v4. The system implements role-based access control (RBAC) with 15 distinct user profiles, ranging from the platform Super Administrator to students, admission candidates, and alumni. The developed modules include: academic management (courses, classes, subjects, enrollments, grades, and attendance), financial management (tuition fees, billing, and payments), academic registry services, communication, final course projects, and internship management. Code quality is ensured through strict TypeScript mode, ESLint, Husky pre-commit hooks, and automated testing with Jest and Cypress. The results demonstrate the technical and economic feasibility of the solution, which operates in a production environment serving real institutions, validating its applicability beyond the academic context.

Keywords: educational management; SaaS; multi-tenant; NestJS; React; TypeScript; RBAC; software engineering.

LISTA DE ABREVIATURAS E SIGLAS

ABNT	Associação Brasileira de Normas Técnicas
ADR	Architecture Decision Record
API	Application Programming Interface
CRUD	Create, Read, Update, Delete
CSS	Cascading Style Sheets
DDD	Domain-Driven Design
DTO	Data Transfer Object
E2E	End-to-End
ERP	Enterprise Resource Planning
HTTP	Hypertext Transfer Protocol
IDE	Integrated Development Environment
IES	Instituição de Ensino Superior
INEP	Instituto Nacional de Estudos e Pesquisas Educacionais Anísio Teixeira
JSON	JavaScript Object Notation
JWT	JSON Web Token
LGPD	Lei Geral de Proteção de Dados
NFSe	Nota Fiscal de Serviço Eletrônica
ORM	Object-Relational Mapper
RBAC	Role-Based Access Control
REST	Representational State Transfer
SaaS	Software as a Service
SPA	Single Page Application
SQL	Structured Query Language
TCC	Trabalho de Conclusão de Curso
UI	User Interface
URL	Uniform Resource Locator
UUID	Universally Unique Identifier
UX	User Experience

SUMARIO

1 INTRODUCAO	14
1.1 Justificativa	14
1.2 Objetivos	15
1.2.1 Objetivo Geral	15
1.2.2 Objetivos Especificos.....	15
2 REFERENCIAL TEORICO	17
2.1 Sistemas de Gestao Educacional.....	17
2.2 Software as a Service (SaaS) e Arquitetura Multi-Tenant.....	17
2.3 Engenharia de Software e Padroes Arquiteturais	18
2.3.1 Domain-Driven Design (DDD)	18
2.3.2 Controle de Acesso Baseado em Papeis (RBAC).....	19
2.3.3 Metodologias Ageis	19
2.4 Tecnologias Utilizadas	19
2.4.1 Node.js e NestJS	19
2.4.2 TypeScript.....	20
2.4.3 React e Vite.....	20
2.4.4 PostgreSQL e Prisma ORM.....	20
2.4.5 Redis e BullMQ	20
3 METODOLOGIA	22
3.1 Tipo de Pesquisa	22
3.2 Metodologia de Desenvolvimento	22
3.2.1 Abordagem Agil Adaptada	22
3.2.2 Processo de Desenvolvimento por Funcionalidade	23
3.2.3 Garantia da Qualidade de Software	23
3.3 Ferramentas Utilizadas.....	23

3.4 Universo e Amostra	24
4 LEVANTAMENTO DE REQUISITOS	25
4.1 Requisitos Funcionais	25
4.2 Requisitos Nao-Funcionais	26
4.3 Casos de Uso.....	27
5 ARQUITETURA DO SISTEMA	30
5.1 Visao Geral da Arquitetura	30
5.2 Padroes Arquiteturais.....	30
5.2.1 Domain-Driven Design (DDD)	30
5.2.2 Arquitetura Multi-Tenant (Row-Level Isolation)	30
5.2.3 Controle de Acesso (RBAC).....	30
5.2.4 Arquitetura Orientada a Eventos.....	30
5.3 Decisoes Arquiteturais (ADRs)	31
6 MODELAGEM DO SISTEMA.....	33
6.1 Modelo Entidade-Relacionamento (ER).....	33
6.1.1 Entidades Principais.....	33
6.1.2 Diagrama ER Simplificado	34
6.1.3 Diagrama Completo	35
6.2 Dicionario de Dados	35
7 DIAGRAMAS DO SISTEMA	37
7.1 Diagrama de Casos de Uso	37
7.2 Diagrama de Classes.....	37
7.3 Organizacao dos Arquivos dos Diagramas	37
8 IMPLEMENTACAO	38
8.1 Implementacao do Backend.....	38
8.1.1 Configuracao do Projeto	38
8.1.2 Modulo de Autenticacao	38

8.1.3 Padrao de Modulo de Dominio	38
8.1.4 Isolamento Multi-Tenant	39
8.1.5 Modulos Implementados.....	39
8.2 Implementacao do Frontend	39
8.2.1 Configuracao do Projeto	39
8.2.2 Arquitetura Baseada em Dominios	39
8.2.3 Roteamento e Controle de Acesso	40
8.2.4 Gerenciamento de Estado Global	40
8.2.5 Resultados Obtidos na Interface	40
8.3 Design System NURA	40
8.3.1 Motivacao	40
8.3.2 Tokens de Design.....	41
8.3.3 Sistema de Dark Mode.....	41
8.3.4 Componentes do Design System	41
8.3.5 Impacto e Resultados	42
9 TESTES	43
9.1 Estrategia de Testes	43
9.2 Testes Unitarios (Jest).....	43
9.3 Testes de Integracao (Supertest)	43
9.4 Testes End-to-End (Cypress)	43
9.5 Resultados dos Testes	43
10 RESULTADOS ALCANCADOS	44
10.1 Sistema Desenvolvido.....	44
10.2 Metricas do Projeto.....	44
10.3 Interface do Sistema.....	45
10.4 Validacao em Ambiente Real	45
10.5 Objetivos Alcançados	45

11 CRONOGRAMA.....	47
11.1 Cronograma Planejado.....	47
11.2 Cronograma Realizado	47
12 CONCLUSAO	48
12.1 Consideracoes Finais	48
12.2 Limitacoes.....	48
12.3 Trabalhos Futuros	49
REFERENCIAS.....	51

1 INTRODUCAO

A transformação digital nas instituições de ensino superior tornou-se uma necessidade estratégica, impulsionada pela crescente demanda por eficiência administrativa, experiência qualificada para alunos e professores, e pela necessidade de integração entre os múltiplos processos que compõem o ecossistema educacional. Nesse contexto, os Sistemas de Gestão Educacional (SGE) emergem como ferramentas fundamentais para centralizar, automatizar e otimizar operações que antes demandavam esforço manual e sistemas dispersos.

O mercado brasileiro de tecnologia educacional denominado EdTech vem crescendo de forma expressiva. Segundo o relatório da Abstartups (2023), o Brasil possui mais de 700 startups educacionais, sendo o maior ecossistema EdTech da América Latina. Contudo, a maioria das soluções disponíveis é voltada ao ensino remoto ou a nichos específicos, deixando uma lacuna significativa no segmento de gestão operacional integrada para instituições de ensino superior de médio porte.

O presente trabalho tem como objeto de estudo o desenvolvimento do NURA, uma plataforma SaaS (Software as a Service) multi-tenant de gestão educacional. O sistema propõe a centralização dos processos acadêmicos, administrativos e financeiros de múltiplas instituições de ensino superior em uma única plataforma digital, garantindo isolamento seguro de dados entre organizações por meio de arquitetura multi-tenant com isolamento por linha de banco de dados.

A escolha pelo modelo SaaS decorre da necessidade de reduzir o custo de implantação e manutenção para as instituições clientes, eliminando a necessidade de infraestrutura local e permitindo atualizações centralizadas. O modelo multi-tenant, por sua vez, possibilita que múltiplos clientes (tenants) utilizem a mesma instância da aplicação de forma isolada, tornando o produto economicamente viável e tecnicamente escalável.

Diante do cenário descrito, emerge o seguinte problema de pesquisa: Como uma plataforma SaaS multi-tenant pode centralizar e automatizar os processos de gestão acadêmica de instituições de ensino superior, garantindo isolamento seguro de dados entre organizações e escalabilidade para múltiplos clientes?

1.1 Justificativa

A justificativa para o desenvolvimento do NURA assenta-se em três dimensões complementares: a necessidade do mercado, a relevância acadêmica e a viabilidade técnica da solução proposta.

Do ponto de vista do mercado, pesquisa realizada pelo Instituto Lobo para o Desenvolvimento da Educação (2022) aponta que mais de 60% das instituições de ensino superior privadas no Brasil com menos de 5.000 alunos ainda não possuem um sistema de gestão integrado. Parte dessas instituições utiliza planilhas eletrônicas para controle de frequência e notas, sistemas financeiros desconectados do módulo acadêmico, e portais de comunicação independentes. Essa fragmentação gera retrabalho, inconsistência de dados e dificulta a tomada de decisão baseada em evidências.

As principais soluções existentes no mercado como TOTVS Educacional, Lyceum e RM apresentam alto custo de licenciamento e implantação, tornando-se inacessíveis para instituições de menor porte. O NURA surge como alternativa estratégica: uma plataforma moderna, modular e acessível por assinatura (SaaS), desenvolvida com tecnologias amplamente adotadas pelo mercado e com custo operacional compartilhado entre os clientes pelo modelo multi-tenant.

Do ponto de vista acadêmico, este trabalho contribui para o campo da Engenharia de Software ao documentar o processo completo de desenvolvimento de um sistema de grande escala, desde a concepção arquitetural até a validação em produção. São abordados padrões arquiteturais modernos como Domain-Driven Design (DDD), controle de acesso baseado em papéis (RBAC), isolamento multi-tenant e design system centralizado, temas com crescente relevância na literatura de engenharia de software (FOWLER, 2002; EVANS, 2003).

Do ponto de vista técnico, a solução demonstra a aplicabilidade de tecnologias de ponta, como NestJS, React 18, TypeScript estrito, Prisma ORM e Tailwind CSS v4, em um sistema real, em produção, com múltiplos módulos e usuários reais. A decisão de utilizar TypeScript em modo estrito em todo o código, aliada a testes automatizados com Jest e Cypress, garante uma base de qualidade que vai além do protótipo acadêmico.

1.2 Objetivos

1.2.1 Objetivo Geral

Desenvolver uma plataforma SaaS multi-tenant de gestão educacional, o NURA, capaz de integrar os processos acadêmicos, administrativos e financeiros de instituições de ensino superior em um único sistema digital escalável e seguro.

1.2.2 Objetivos Específicos

1. Projetar uma arquitetura multi-tenant com isolamento de dados por organização utilizando NestJS e Prisma ORM sobre PostgreSQL;

2. Implementar um sistema de controle de acesso baseado em papéis (RBAC) com 15 perfis de usuário distintos, cobrindo todos os atores institucionais;
3. Desenvolver interfaces de usuário responsivas e acessíveis utilizando React 18, TypeScript e Tailwind CSS v4, com suporte nativo a modo claro e escuro;
4. Construir um design system centralizado com tokens de cor, tipografia e componentes reutilizáveis, garantindo consistência visual em toda a plataforma;
5. Integrar funcionalidades de gestão acadêmica, financeira, secretaria, comunicação e trabalhos de conclusão de curso em módulos independentes e escaláveis;
6. Validar o sistema por meio de testes automatizados unitários (Jest), de integração (Supertest) e ponta a ponta (End-to-End) com Cypress;
7. Documentar o processo de desenvolvimento, as decisões arquiteturais e os resultados obtidos, contribuindo para a literatura técnica e acadêmica do campo da Engenharia de Software.

2 REFERENCIAL TEORICO

2.1 Sistemas de Gestão Educacional

Os Sistemas de Gestão Educacional (SGE), também denominados Academic Management Systems (AMS) na literatura internacional, são plataformas de software concebidas para integrar e automatizar os processos administrativos, acadêmicos e financeiros de instituições de ensino.

Segundo Nakayama, Pilla e Rissi (2016), esses sistemas emergiram da evolução dos Sistemas de Informação Gerencial (SIG) aplicados ao contexto educacional, incorporando progressivamente módulos de secretaria, financeiro, biblioteca, estágio e comunicação.

No contexto brasileiro, a regulamentação das Instituições de Ensino Superior (IES) pelo Ministério da Educação (MEC), especialmente quanto ao Sistema Nacional de Avaliação da Educação Superior (SINAES) e ao Censo da Educação Superior conduzido pelo INEP, impõe as instituições a necessidade de manter registros detalhados e auditáveis de todos os seus processos acadêmicos. Essa demanda regulatória torna a adoção de sistemas de gestão integrada não apenas conveniente, mas essencialmente estratégia para a sobrevivência institucional.

Sommerville (2018) classifica os sistemas de informação institucionais em dois grandes grupos: sistemas transacionais, que registram e processam operações do dia a dia, e sistemas de suporte a decisão, que agregam e analisam dados para apoiar a gestão estratégica. Os SGEs modernos devem contemplar ambas as dimensões, oferecendo ao mesmo tempo eficiência operacional e visibilidade analítica para gestores educacionais.

A literatura registra uma evolução marcante nos modelos de entrega desses sistemas. Soluções como TOTVS Educacional, Lyceum e Moodle, amplamente utilizadas no Brasil, adotam o modelo de licença tradicional, com instalação on-premise, elevado custo inicial e necessidade de infraestrutura própria. Tal modelo apresenta barreiras significativas para instituições de menor porte, criando um segmento de mercado carente de soluções mais acessíveis e modernas (LOBO; LOBO, 2022).

2.2 Software as a Service (SaaS) e Arquitetura Multi-Tenant

O modelo de distribuição de software denominado Software as a Service (SaaS) representa uma evolução significativa em relação ao paradigma tradicional de licenciamento. Neste modelo, o software é hospedado pelo fornecedor e disponibilizado aos clientes por meio da internet, geralmente mediante assinatura periódica. Conforme definição do National Institute of Standards and Technology (NIST, 2011), SaaS é uma das três camadas fundamentais da

computação em nuvem, ao lado de Platform as a Service (PaaS) e Infrastructure as a Service (IaaS).

As principais vantagens do modelo SaaS para os clientes incluem: eliminação do custo de infraestrutura local, atualizações automáticas e centralizadas, escalabilidade elástica conforme demanda e acessibilidade a partir de qualquer dispositivo com acesso à internet (MELL; GRANCE, 2011). Para o desenvolvedor, o modelo permite um fluxo de receita recorrente e previsível, além de facilitar o ciclo de melhorias contínuas.

A arquitetura multi-tenant é o padrão predominante em sistemas SaaS, permitindo que uma única instância da aplicação sirva múltiplos clientes denominados tenants de forma logicamente isolada. Bezemer e Zaidman (2010) identificam três estratégias principais de isolamento de dados em arquiteturas multi-tenant:

Banco de dados separado por tenant (database-per-tenant): cada cliente possui seu próprio banco de dados. Oferece o maior grau de isolamento e segurança, mas impõe alto custo operacional e complexidade de manutenção conforme o número de clientes cresce.

Schema separado por tenant (schema-per-tenant): todos os tenants compartilham o mesmo servidor de banco de dados, mas cada um possui seu próprio schema. Equilíbrio razoável entre isolamento e eficiência, porém com complexidade de migração de dados.

Isolamento por linha (row-level isolation): todos os tenants compartilham as mesmas tabelas, com um campo identificador, geralmente `organizacaoId`, distinguindo os registros de cada cliente. É a abordagem mais eficiente em termos de recursos computacionais, sendo amplamente adotada em sistemas SaaS de médio porte (WEISSMAN; BOBROWSKI, 2009).

O NURA adota a estratégia de isolamento por linha, com o campo `organizacaoId` presente em todas as entidades do sistema. Um middleware de contexto, o `TenantMiddleware`, intercepta cada requisição HTTP, extrai o identificador da organização do token JWT e o injeta automaticamente em todas as operações de banco de dados, garantindo que nenhum usuário acesse inadvertidamente dados de outra organização.

2.3 Engenharia de Software e Padrões Arquiteturais

2.3.1 Domain-Driven Design (DDD)

Domain-Driven Design (DDD) é uma abordagem de desenvolvimento de software proposta por Evans (2003) que coloca o domínio de negócio no centro das decisões arquiteturais. O DDD preconiza que o código deve refletir fielmente a linguagem e os conceitos

do negócio, a chamada ubiquitous language, e que o sistema deve ser organizado em torno de contextos delimitados (bounded contexts) que encapsulam a lógica de cada subdomínio.

No NURA, o DDD se manifesta na organização modular tanto do backend quanto do frontend. O backend é dividido em módulos NestJS que espelham os domínios de negócio: acadêmico, financeiro, pessoas, secretaria, comunicação, tcc, estágio, entre outros. Cada módulo é auto contido, com seus próprios controllers, services, DTOs e entidades, reduzindo o acoplamento entre domínios e facilitando a evolução independente de cada área.

2.3.2 Controle de Acesso Baseado em Papeis (RBAC)

O modelo de controle de acesso baseado em papéis, Role-Based Access Control (RBAC), foi formalizado por Sandhu et al. (1996) e tornou-se o padrão dominante em sistemas empresariais. No RBAC, permissões são associadas a papéis (roles), e usuários recebem papéis que determinam suas capacidades no sistema. Essa abordagem simplifica a administração de permissões em sistemas com muitos usuários e granularidade de acesso complexa.

O NURA implementa um modelo RBAC com 15 papéis distintos: SUPER_ADMIN, ADMIN, COORDENADOR, PROFESSOR, SECRETARIA, FINANCEIRO, RH, BIBLIOTECARIO, PORTEIRO, OUVIDORIA, ASSESSORIA_ACADEMICA, ALUNO, RESPONSAVEL, EGRESSO e CANDIDATO_PS. O controle é implementado por meio de decoradores NestJS (@Papeis()) e um guard customizado (PapeisGuard) que valida o papel do usuário autenticado antes de conceder acesso a cada endpoint.

2.3.3 Metodologias Ágeis

O Manifesto Ágil (BECK et al., 2001) estabeleceu os princípios que norteiam as metodologias de desenvolvimento iterativo e incremental. Frameworks como Scrum e Kanban derivam desses princípios, enfatizando entregas frequentes, colaboração entre equipe e adaptação contínua às mudanças. Pressman e Maxim (2016) destacam que as metodologias ágeis são especialmente adequadas para projetos com requisitos em evolução e equipes enxutas, características presentes no desenvolvimento do NURA.

2.4 Tecnologias Utilizadas

2.4.1 Node.js e NestJS

Node.js é um ambiente de execução JavaScript do lado do servidor, baseado no motor V8 do Google Chrome. Seu modelo de I/O não-bloqueante e orientado a eventos o torna particularmente eficiente para APIs REST com alto volume de requisições concorrentes

(TILKOV; VINOSKI, 2010). O NestJS é um framework progressivo para Node.js que utiliza TypeScript nativamente e é fortemente inspirado na arquitetura do Angular, fornecendo suporte a módulos, injeção de dependências, decoradores e guards de forma estruturada e testável.

2.4.2 TypeScript

TypeScript é um superconjunto tipado do JavaScript, desenvolvido pela Microsoft, que adiciona tipagem estática opcional a linguagem. A tipagem estática permite a detecção de erros em tempo de compilação, antes mesmo da execução, melhora a documentação implícita do código e viabiliza ferramentas de refactoring mais poderosas (HEJLSBERG et al., 2023). No NURA, TypeScript é utilizado em modo estrito (`strict: true`) em todo o projeto, com zero tolerância ao tipo `any`, garantido por regras ESLint e pre-commit hooks.

2.4.3 React e Vite

React é uma biblioteca JavaScript declarativa para construção de interfaces de usuário, desenvolvida e mantida pela Meta. Seu modelo baseado em componentes reutilizáveis e seu sistema de reconciliação virtual de DOM (Virtual DOM) tornaram-no a escolha dominante para o desenvolvimento de SPAs (Single Page Applications) no ecossistema JavaScript (REACT, 2024). O Vite é um bundler de nova geração que utiliza ES modules nativos do navegador durante o desenvolvimento, proporcionando tempos de inicialização e hot reload significativamente superiores aos bundlers anteriores.

2.4.4 PostgreSQL e Prisma ORM

PostgreSQL é um sistema de gerenciamento de banco de dados objeto-relacional de código aberto, reconhecido por sua robustez, extensibilidade, conformidade com o padrão SQL e suporte a tipos de dados avançados. O Prisma é um ORM moderno para Node.js e TypeScript que oferece type-safety completa nas operações de banco de dados: o schema Prisma e a fonte única de verdade, gerando automaticamente os tipos TypeScript correspondentes, eliminando discrepâncias entre o modelo de dados e o código de aplicação.

2.4.5 Redis e BullMQ

Redis é um armazenamento de estrutura de dados em memória, frequentemente utilizado como cache, session store e broker de mensagens. O BullMQ é uma biblioteca para processamento de filas baseada em Redis, amplamente adotada no ecossistema NestJS para gerenciamento de tarefas assíncronas. No NURA, o BullMQ é utilizado para envio assíncrono

de e-mails e notificações, desacoplando essas operações do ciclo de vida da requisição HTTP e garantindo que falhas no envio não impactem a resposta ao usuário.

3 METODOLOGIA

3.1 Tipo de Pesquisa

Este trabalho caracteriza-se como uma pesquisa aplicada, uma vez que busca gerar conhecimento voltado para a solução de um problema prático identificado no contexto das instituições de ensino superior, especificamente a fragmentação dos sistemas de gestão acadêmica, administrativa e financeira.

Quanto aos objetivos, a pesquisa possui caráter exploratório e descritivo. A natureza exploratória está relacionada ao levantamento e estudo de conceitos presentes na literatura sobre arquitetura de software, sistemas SaaS, engenharia de software e sistemas de gestão educacional. Já o caráter descritivo manifesta-se na documentação detalhada do processo de desenvolvimento do sistema, das decisões arquiteturais adotadas e dos resultados obtidos ao longo do projeto.

Em relação aos procedimentos técnicos, adotou-se o estudo de caso como estratégia principal de pesquisa. Segundo Yin (2015), o estudo de caso consiste em uma investigação empírica que analisa um fenômeno contemporâneo dentro de seu contexto real, sendo especialmente adequado quando os limites entre o fenômeno e o contexto não são claramente definidos. Nesse sentido, o objeto de estudo corresponde ao sistema NURA, concebido, desenvolvido e validado em ambiente real de utilização.

Complementarmente, foram utilizados procedimentos de pesquisa bibliográfica, fundamentados em obras relacionadas a engenharia de software, arquitetura de sistemas, metodologias ágeis e tecnologias empregadas no desenvolvimento da solução. Além disso, a pesquisa apresenta características experimentais, considerando a implementação iterativa do sistema e a realização contínua de testes para validação dos componentes desenvolvidos.

A abordagem metodológica adotada é predominantemente qualitativa, concentrando-se na análise das decisões arquiteturais, dos desafios encontrados e das soluções implementadas. Entretanto, indicadores quantitativos, como cobertura de testes, quantidade de módulos implementados e desempenho da API, foram utilizados de forma complementar para subsidiar as análises realizadas.

3.2 Metodologia de Desenvolvimento

3.2.1 Abordagem Ágil Adaptada

O desenvolvimento do sistema NURA foi conduzido por uma equipe composta por dois desenvolvedores, utilizando uma abordagem ágil adaptada as características e necessidades do projeto. Foram incorporados princípios provenientes das metodologias Scrum e Kanban, priorizando ciclos curtos de desenvolvimento, entregas incrementais e melhoria contínua.

As iterações ocorreram em períodos de uma a duas semanas, ao final dos quais novas funcionalidades eram integradas ao ambiente compartilhado de desenvolvimento e submetidas a validação conjunta pelos integrantes da equipe. O GitHub foi utilizado como plataforma de versionamento, revisão de código e acompanhamento das atividades realizadas.

3.2.2 Processo de Desenvolvimento por Funcionalidade

O processo de implementação das funcionalidades seguiu uma sequência padronizada composta pelas seguintes etapas: levantamento e análise dos requisitos funcionais e das regras de negócio; desenvolvimento da camada de backend utilizando o framework NestJS; desenvolvimento da interface frontend utilizando React, Redux e integração com a API por meio do Axios; realização de testes automatizados unitários e testes de ponta a ponta para validação dos fluxos críticos; revisão do código-fonte por meio de Pull Requests; integração e validação em ambiente compartilhado de desenvolvimento.

3.2.3 Garantia da Qualidade de Software

A qualidade do software foi assegurada por meio da utilização de ferramentas e práticas automatizadas que atuam em diferentes etapas do ciclo de desenvolvimento.

Camada	Ferramenta	Momento de Execução
Tipagem estática	TypeScript (Strict Mode)	Durante o desenvolvimento
Análise de código	ESLint	Processo de pre-commit
Verificação de compilação	TypeScript Compiler (tsc)	Processo de pre-commit
Testes unitários	Jest	Pre-push e integração contínua
Testes E2E	Cypress	Integração contínua
CI/CD	GitHub Actions	Publicação na branch principal

3.3 Ferramentas Utilizadas

As principais ferramentas empregadas durante o desenvolvimento do sistema são apresentadas na tabela a seguir.

Categoria	Ferramenta	Finalidade
Versionamento	Git e GitHub	Controle de versões e colaboração
IDE	Visual Studio Code	Implementação e depuração
Testes de API	Insomnia e Swagger UI	Validação de endpoints
Banco de Dados	DBeaver	Administração do PostgreSQL
Containers	Docker e Docker Compose	Ambiente padronizado
Modelagem	Draw.io	Criação de diagramas UML
Documentação	Markdown	Documentação técnica
CI/CD	GitHub Actions e PM2	Automação de deploy
Comunicação	WhatsApp e GitHub Issues	Coordenação de atividades

3.4 Universo e Amostra

O sistema NURA foi desenvolvido com foco em instituições de ensino superior de médio porte, especialmente faculdades e centros universitários privados com até 10.000 estudantes matriculados.

Os testes e validações ocorreram em ambientes distintos de desenvolvimento e produção, possibilitando a avaliação do comportamento do sistema tanto em cenários simulados quanto em situações reais de utilização.

A validação junto a gestores, coordenadores e profissionais administrativos de instituições parceiras forneceu subsídios importantes para o refinamento dos módulos implementados. Dessa forma, a amostra utilizada caracteriza-se como não probabilística por conveniência, sendo composta pelos usuários que participaram voluntariamente do processo de avaliação da solução desenvolvida.

4 LEVANTAMENTO DE REQUISITOS

4.1 Requisitos Funcionais

Os requisitos funcionais descrevem o que o sistema deve fazer, ou seja, as funcionalidades e comportamentos esperados.

Modulo: Autenticação e Acesso

Código	Requisito	Prioridade
RF001	O sistema deve permitir login com e-mail e senha	Alta
RF002	O sistema deve emitir token JWT após autenticação bem-sucedida	Alta
RF003	O sistema deve suportar redefinição de senha por e-mail	Alta
RF004	O sistema deve implementar 15 papéis de usuário distintos	Alta
RF005	O sistema deve isolar dados por organização (multi-tenant)	Alta

Modulo: Gestão Acadêmica

Código	Requisito	Prioridade
RF010	O sistema deve permitir cadastro e gestão de cursos	Alta
RF011	O sistema deve permitir cadastro e gestão de turmas	Alta
RF012	O sistema deve permitir cadastro e gestão de disciplinas	Alta
RF013	O sistema deve permitir matrícula de alunos em turmas	Alta
RF014	O sistema deve registrar frequência e notas por disciplina	Alta
RF015	O sistema deve gerar histórico escolar do aluno	Média

Modulo: Pessoas

Código	Requisito	Prioridade
RF020	O sistema deve permitir cadastro de alunos	Alta
RF021	O sistema deve permitir cadastro de professores	Alta
RF022	O sistema deve permitir cadastro de funcionários	Média
RF023	O sistema deve gerenciar documentação dos alunos	Média

Modulo: Financeiro

Código	Requisito	Prioridade
RF030	O sistema deve gerar cobranças de mensalidades	Alta
RF031	O sistema deve registrar pagamentos realizados	Alta
RF032	O sistema deve gerar relatórios financeiros	Média
RF033	O sistema deve suportar geração de NFSe	Baixa

Modulo: TCC

Código	Requisito	Prioridade
RF040	O sistema deve permitir cadastro de projetos de TCC	Alta
RF041	O sistema deve associar orientador ao projeto de TCC	Alta
RF042	O sistema deve registrar etapas e prazos do TCC	Média
RF043	O sistema deve permitir upload de documentos do TCC	Média

4.2 Requisitos Não-Funcionais

Os requisitos não-funcionais descrevem como o sistema deve se comportar em termos de qualidades.

Desempenho

Código	Requisito
RNF001	O sistema deve responder a requisições de API em até 500ms em condições normais
RNF002	O sistema deve suportar ao menos 100 usuários simultâneos sem degradação perceptível
RNF003	O frontend deve ter tempo de carregamento inicial inferior a 3 segundos

Segurança

Código	Requisito
RNF010	Os tokens JWT devem ter expiração configurável (padrão: 7 dias)
RNF011	Senhas devem ser armazenadas com hash bcrypt (fator 10+)
RNF012	Toda comunicação deve ocorrer via HTTPS em produção

RNF013	O sistema deve implementar isolamento completo de dados entre organizações
RNF014	O sistema deve registrar logs de auditoria para ações sensíveis

Usabilidade

Código	Requisito
RNF020	A interface deve ser responsiva (desktop, tablet, mobile)
RNF021	O sistema deve suportar modo claro e escuro (light/dark mode)
RNF022	O design deve seguir padrões de acessibilidade (WCAG 2.1 nível AA)

Manutenibilidade

Código	Requisito
RNF030	O código deve ter cobertura de testes mínima de 70% (backend)
RNF031	O sistema deve usar TypeScript em modo estrito (zero any)
RNF032	Toda feature nova deve seguir o padrão de modulo definido na documentação

Disponibilidade e Confiabilidade

Código	Requisito
RNF040	O sistema deve ter uptime de 99,5% em ambiente de produção
RNF041	O sistema deve ter estratégia de backup do banco de dados
RNF042	O deploy deve ser automatizado via CI/CD (GitHub Actions)

4.3 Casos de Uso

Os casos de uso descrevem as interações entre os atores e o sistema.

Atores do Sistema

Ator	Descrição
Super Admin	Administrador global da plataforma (acesso a todas as organizacoes)
Admin	Administrador de uma organização específica
Coordenador	Coordenador de curso

Professor	Docente da instituição
Aluno	Estudante matriculado
Secretaria	Atendente da secretaria acadêmica
Financeiro	Responsável pelo setor financeiro

UC001 - Realizar Login

Ator: Qualquer usuário cadastrado

Pré-condição: Usuário deve ter conta ativa no sistema

Fluxo Principal: 1) Usuário acessa a tela de login; 2) Usuário informa e-mail e senha; 3) Sistema valida as credenciais; 4) Sistema emite token JWT; 5) Sistema redireciona para o dashboard conforme papel do usuário.

Fluxo Alternativo (3a): Credenciais invalidas - sistema exibe mensagem de erro.

Fluxo Alternativo (3b): Conta inativa - sistema exibe mensagem de organização inativa.

UC002 - Realizar Matrícula de Aluno

Ator: Secretaria, Admin

Pré-condição: Aluno e turma devem estar cadastrados

Fluxo Principal: 1) Usuário busca o aluno pelo nome ou CPF; 2) Usuário seleciona a turma/curso desejado; 3) Sistema verifica disponibilidade de vagas; 4) Sistema registra a matrícula; 5) Sistema envia confirmação por e-mail ao aluno.

UC003 - Lançar Notas e Frequência

Ator: Professor

Pré-condição: Professor deve estar vinculado a disciplina/turma

Fluxo Principal: 1) Professor acessa o diário de classe; 2) Professor seleciona a turma e a disciplina; 3) Professor lança presença dos alunos; 4) Professor lançar notas das avaliações; 5) Sistema calcula media e situação do aluno (aprovado/reprovado).

UC004 - Gerar Histórico Escolar

Ator: Aluno, Secretaria

Pré-condição: Aluno deve ter disciplinas cursadas registradas

Fluxo Principal: 1) Usuário solicita o histórico escolar; 2) Sistema consolida todas as disciplinas cursadas com notas e frequências; 3) Sistema gera documento PDF com o histórico; 4) Sistema disponibiliza para download.

5 ARQUITETURA DO SISTEMA

5.1 Visão Geral da Arquitetura

O NURA adota uma arquitetura cliente-servidor organizada em monorepo, com clara separação entre a camada de apresentação (frontend SPA) e a camada de negócio e dados (backend API REST).

A camada de apresentação é construída com React 18, TypeScript e Redux Toolkit, operando como uma Single Page Application (SPA). A comunicação entre frontend e backend ocorre por meio de requisições HTTP REST com payloads em formato JSON, sendo a autenticação realizada via tokens JWT.

A camada de negócio é implementada com NestJS e TypeScript, contendo mais de 30 módulos de domínio que cobrem áreas como acadêmico, financeiro, pessoas, autenticação, entre outros. O acesso ao banco de dados é mediado pelo Prisma ORM, enquanto o Redis é utilizado para cache e gerenciamento de filas de tarefas assíncronas via BullMQ.

A camada de dados é composta pelo PostgreSQL como sistema gerenciador de banco de dados principal e pelo Redis como armazenamento em memória para cache e filas.

5.2 Padrões Arquiteturais

5.2.1 Domain-Driven Design (DDD)

O sistema organiza seu código em torno de domínios de negócio (Acadêmico, Financeiro, Pessoas etc.), cada um com sua própria camada de controller, service e repository. Isso facilita a manutenção e evolução independente de cada domínio.

5.2.2 Arquitetura Multi-Tenant (Row-Level Isolation)

Cada registro no banco de dados possui um campo `organizacaoId` que garante o isolamento entre organizações. O `TenantMiddleware` intercepta todas as requisições e injeta automaticamente o contexto da organização corrente.

5.2.3 Controle de Acesso (RBAC)

O sistema implementa Role-Based Access Control com 15 papéis distintos. O acesso a cada endpoint é controlado por decoradores (`@Papeis()`) e guards NestJS.

5.2.4 Arquitetura Orientada a Eventos

Para operações assíncronas como envio de e-mails e notificações push, o sistema utiliza filas BullMQ sobre Redis, desacoplando o processamento assíncrono da requisição HTTP.

5.3 Decisões Arquiteturais (ADRs)

As decisões arquiteturais foram documentadas no formato Architecture Decision Record (ADR), que registra o contexto, as opções consideradas, a decisão tomada e suas consequências.

ADR-001 - Isolamento Multi-Tenant via Row-Level

Data: Abril 2026 | Status: Aceito

Contexto: O NURA precisa servir múltiplas instituições com total isolamento de dados. Foram avaliadas três opções: database-per-tenant, schema-per-tenant e row-level isolation. Decisão: Row-level isolation pela simplicidade operacional e adequação ao volume esperado de tenants. Consequências: Todo service backend deve filtrar dados por organizacaoId. O TenantMiddleware garante essa injeção automática via contexto da requisição.

ADR-002 - Tailwind CSS v4 como Framework de Estilizacao

Data: Abril 2026 | Status: Aceito

Contexto: O frontend precisa de um sistema de estilização consistente, com suporte nativo a dark mode e tokens centralizados. Decisão: Tailwind CSS v4 com tokens definidos em variáveis CSS no arquivo index.css. Consequências: Disciplina rigorosa necessária, nenhum desenvolvedor pode usar cores hardcoded.

ADR-003 - Gaveta (Drawer) como Padrão de UI para Formulários

Data: Abril 2026 | Status: Aceito

Contexto: O sistema possui dezenas de formulários de criação e edição. Decisão: Gaveta (drawer lateral, abrindo da direita) como padrão preferencial. Modal apenas para confirmações simples. Justificativa: O drawer preserva o contexto da tela anterior e padroniza o fluxo lateral da aplicação.

ADR-004 - TypeScript Strict Mode (Zero any)

Data: Abril 2026 | Status: Aceito

Contexto: Garantir qualidade e manutenibilidade do código em um projeto de grande escala. Decisão: TypeScript em modo estrito em todo o projeto. O uso de any é proibido e

verificado automaticamente por ESLint e Husky (pre-commit hooks). Consequências: Maior verbosidade inicial no desenvolvimento, mas detecção precoce de erros e documentação implícita dos contratos de dados.

6 MODELAGEM DO SISTEMA

6.1 Modelo Entidade-Relacionamento (ER)

O banco de dados do sistema NURA foi implementado utilizando o Sistema Gerenciador de Banco de Dados PostgreSQL, sendo sua estrutura gerenciada por meio do Prisma ORM. O esquema de dados encontra-se definido no diretório responsável pela modelagem do banco de dados da aplicação, permitindo a representação das entidades, atributos e relacionamentos necessários ao funcionamento do sistema.

6.1.1 Entidades Principais

Organização (Tenant)

A entidade Organização representa uma instituição de ensino cadastrada na plataforma. Em razão da arquitetura multi-tenant adotada pelo sistema, todas as demais entidades possuem relacionamento direto ou indireto com essa entidade, garantindo o isolamento e a segurança dos dados entre diferentes organizações.

Campo	Tipo	Descrição
id	String (UUID)	Identificador único da organização
nome	String	Nome da instituição
slug	String	Identificador único utilizado na URL
ativa	Boolean	Indica se a organização está ativa
criadaEm	DateTime	Data de criação do registro

Usuário

A entidade Usuário representa qualquer indivíduo que utiliza o sistema, incluindo administradores, professores, alunos e demais perfis cadastrados.

Campo	Tipo	Descrição
id	String (UUID)	Identificador único do usuário
email	String	Endereço eletrônico do usuário
senhaHash	String	Senha armazenada de forma criptografada
papel	Enum	Perfil de acesso do usuário
organizacaoId	String	Chave estrangeira para Organização
ativo	Boolean	Indica se o usuário está ativo

Curso

A entidade Curso armazena as informações referentes aos cursos disponibilizados por cada instituição de ensino.

Campo	Tipo	Descrição
id	String (UUID)	Identificador único do curso
nome	String	Nome do curso
cargaHoraria	Int	Carga horaria total do curso
organizacaoId	String	Chave estrangeira para Organização

Turma

A entidade Turma representa os agrupamentos de alunos vinculados a um determinado curso.

Campo	Tipo	Descrição
id	String (UUID)	Identificador único da turma
nome	String	Nome da turma
cursoId	String	Chave estrangeira para Curso
organizacaoId	String	Chave estrangeira para Organização

Matrícula

A entidade Matrícula é responsável por registrar o vínculo entre alunos e turmas, permitindo o controle academico dos participantes.

Campo	Tipo	Descrição
id	String (UUID)	Identificador único da matrícula
alunoId	String	Chave estrangeira para Usuário
turmaId	String	Chave estrangeira para Turma
dataMatricula	DateTime	Data de realização da matrícula
status	Enum	Situação da matrícula (ATIVA, TRANCADA ou CANCELADA)

6.1.2 Diagrama ER Simplificado

O relacionamento entre as principais entidades do sistema pode ser representado de forma simplificada. Observa-se que uma organização pode possuir diversos usuários e cursos.

Cada curso pode conter várias turmas, enquanto as matrículas estabelecem o relacionamento entre alunos e turmas. Além disso, o sistema contempla módulos complementares relacionados a gestão financeira e ao gerenciamento de Trabalhos de Conclusão de Curso (TCC).

6.1.3 Diagrama Completo

O diagrama entidade-relacionamento completo do sistema NURA contém todas as entidades implementadas, seus respectivos atributos e relacionamentos. O diagrama completo está disponível no diretório de diagramas do projeto.

6.2 Dicionário de Dados

O dicionário de dados tem como finalidade documentar detalhadamente a estrutura do banco de dados do sistema NURA, apresentando as entidades, atributos, tipos de dados, obrigatoriedade e descrição funcional de cada campo.

Tabela: Organização

A tabela Organização armazena os dados das instituições de ensino cadastradas na plataforma, sendo a entidade central da arquitetura multi-tenant adotada pelo sistema.

Campo	Tipo	Obrigatório	Descrição
id	UUID	Sim	Chave primaria gerada automaticamente
nome	String	Sim	Nome da instituição de ensino
slug	String	Sim	Identificador único utilizado em URLs
ativa	Boolean	Sim	Indica se a organização está ativa
logoUrl	String	Nao	Endereço eletrônico da logomarca
criadaEm	DateTime	Sim	Data e hora de criação do registro
atualizadaEm	DateTime	Sim	Data e hora da última atualização

Tabela: Usuário

A tabela Usuário concentra os dados dos indivíduos que acessam o sistema, independentemente do perfil de utilização.

Campo	Tipo	Obrigatório	Descrição
id	UUID	Sim	Chave primaria
email	String	Sim	Endereço de e-mail único no sistema

senhaHash	String	Sim	Senha armazenada de forma criptografada
papel	Enum	Sim	Perfil de acesso do usuário
organizacaoId	UUID	Sim	Chave estrangeira para Organização
nome	String	Sim	Nome completo do usuário
ativo	Boolean	Sim	Indica se a conta está ativa
criadoEm	DateTime	Sim	Data e hora de criação do registro

Enumeracao: PapelUsuario

O campo papel utiliza uma enumeracao responsável por definir os níveis de acesso e responsabilidades dentro do sistema: SUPER_ADMIN, ADMIN, COORDENADOR, PROFESSOR, SECRETARIA, FINANCEIRO, RH, BIBLIOTECARIO, PORTEIRO, OUVIDORIA, ASSESSORIA_ACADEMICA, ALUNO, RESPONSVEL, EGRESSO e CANDIDATO_PS.

Tabela: Curso

Campo	Tipo	Obrigatório	Descrição
id	UUID	Sim	Chave primaria
nome	String	Sim	Nome do curso
cargaHoraria	Integer	Sim	Carga horaria total em horas
organizacaoId	UUID	Sim	Chave estrangeira para Organização
ativo	Boolean	Sim	Indica se o curso esta ativo

O sistema NURA possui outras entidades relacionadas a gestão acadêmica, incluindo Turma, Matricula, Disciplina, Nota, Frequência, Financeiro e módulos específicos para acompanhamento de Trabalhos de Conclusão de Curso (TCC). O detalhamento dessas entidades segue a mesma estrutura apresentada anteriormente.

7 DIAGRAMAS DO SISTEMA

Este capítulo apresenta os principais diagramas utilizados para representar a estrutura, o comportamento e os fluxos do sistema NURA. Os diagramas foram elaborados utilizando a linguagem UML (Unified Modeling Language), com o objetivo de facilitar a compreensão da arquitetura do sistema, dos relacionamentos entre seus componentes e dos processos executados pelos usuários.

7.1 Diagrama de Casos de Uso

O Diagrama de Casos de Uso tem como finalidade representar as principais funcionalidades disponibilizadas pelo sistema e os atores que interagem com cada uma delas. Os principais atores identificados são: Super Administrador, Administrador, Professor, Aluno e Secretaria. As principais funcionalidades contempladas incluem: realização de login, gerenciamento de organizações, lançamento de notas, consulta de histórico acadêmico e realização de matrículas.

7.2 Diagrama de Classes

O Diagrama de Classes representa a estrutura estática do sistema, evidenciando as principais entidades, seus atributos, métodos e relacionamentos. As principais classes modeladas no sistema NURA incluem: Organizacao, Usuario, Curso, Turma, Matricula, Disciplina, Nota, Financeiro e Projeto TCC.

7.3 Organização dos Arquivos dos Diagramas

Para fins de documentação e manutenção do projeto, todos os diagramas produzidos são armazenados no diretório destinado a documentação técnica do sistema.

Arquivo	Conteúdo
diagrama-er.png	Modelo Entidade-Relacionamento completo
diagrama-casos-de-uso.png	Diagrama UML de Casos de Uso
diagrama-classes.png	Diagrama UML de Classes
diagrama-sequencia-login.png	Diagrama de Sequência do processo de Login
diagrama-arquitetura.png	Visão geral da arquitetura do sistema

8 IMPLEMENTACAO

8.1 Implementação do Backend

A camada de backend do sistema NURA foi desenvolvida utilizando o framework NestJS, uma plataforma baseada em Node.js que adota conceitos de modularização, injeção de dependência e arquitetura orientada a serviços. O backend é responsável pelo processamento das regras de negócio, autenticação de usuários, controle de permissões, integração com o banco de dados e disponibilização dos serviços consumidos pelo frontend por meio de uma API REST.

8.1.1 Configuração do Projeto

O projeto foi inicializado utilizando a ferramenta oficial NestJS CLI, sendo configurado com TypeScript em modo estrito (strict mode), visando aumentar a segurança do código e reduzir falhas durante o desenvolvimento. A estrutura modular adotada permite a criação de novos domínios de negócio sem impactar os módulos já existentes, favorecendo a manutenção e evolução contínua da aplicação.

8.1.2 Modulo de Autenticação

O processo de autenticação foi implementado utilizando o padrão Bearer Token baseado em JSON Web Token (JWT). O fluxo de autenticação ocorre conforme as etapas: o usuário realiza uma requisição para o endpoint de autenticação; o sistema valida as credenciais informadas; após a validação, é gerado um token JWT contendo informações do usuário autenticado; o token é enviado ao cliente e armazenado para utilização nas requisições subsequentes; as rotas protegidas validam automaticamente o token recebido.

O token gerado contém informações essenciais para identificação do usuário e do contexto organizacional, incluindo: identificador do usuário, endereço de e-mail, perfil de acesso e identificador da organização. A proteção das rotas é realizada por meio de Guards do NestJS, enquanto o contexto organizacional é definido por um middleware responsável por identificar a organização associada a requisição.

8.1.3 Padrão de Modulo de Domínio

Cada domínio de negócio foi desenvolvido seguindo o padrão modular recomendado pelo NestJS. Cada modulo é composto por: Controllers, responsáveis pelo recebimento das requisições HTTP; Services, responsáveis pela implementação das regras de negócio; DTOs (Data Transfer Objects), responsáveis pela validação e transferência de dados; e Providers e dependências compartilhadas.

8.1.4 Isolamento Multi-Tenant

Um dos requisitos fundamentais do sistema NURA consiste na utilização da arquitetura multi-tenant, permitindo que múltiplas instituições utilizem a mesma infraestrutura sem compartilhamento indevido de dados. Para garantir esse isolamento, todas as consultas realizadas ao banco de dados consideram o identificador da organização associada ao usuário autenticado. Essa estratégia assegura que cada instituição visualize exclusivamente suas próprias informações.

8.1.5 Módulos Implementados

O backend do NURA foi desenvolvido de forma modular, contemplando diferentes áreas da gestão educacional. Entre os principais módulos implementados destacam-se: autenticação e autorização, gestão de usuários, gestão de organizações, gestão acadêmica, matrículas, controle de cursos e turmas, gestão financeira, biblioteca, gestão de Trabalhos de Conclusão de Curso (TCC), relatórios gerenciais e configurações institucionais.

A implementação modular contribuiu para a escalabilidade do sistema, permitindo a inclusão de novas funcionalidades sem necessidade de alterações significativas nos componentes já existentes.

8.2 Implementação do Frontend

A camada de frontend do sistema NURA foi desenvolvida utilizando React 18, com suporte ao TypeScript para tipagem estática e maior segurança durante o desenvolvimento. A aplicação possui arquitetura modular baseada em domínios de negócio, permitindo escalabilidade, reutilização de componentes e facilidade de manutenção.

8.2.1 Configuração do Projeto

O projeto frontend foi inicializado utilizando a ferramenta Vite, escolhida por oferecer maior desempenho durante o desenvolvimento e geração otimizada dos arquivos para produção. Além do React e TypeScript, foram utilizadas bibliotecas para gerenciamento de estado global, roteamento, integração com APIs e estilização da interface.

8.2.2 Arquitetura Baseada em Domínios

A organização do código segue uma arquitetura orientada a domínios de negócio, espelhando a estrutura existente no backend. Dessa forma, cada módulo do sistema possui seus próprios componentes, serviços, estados e rotas. Essa abordagem favorece a modularização da

aplicação, reduzindo o acoplamento entre funcionalidades e facilitando a manutenção do código ao longo do ciclo de vida do projeto.

8.2.3 Roteamento e Controle de Acesso

O gerenciamento das rotas foi implementado utilizando React Router. O sistema suporta carregamento sob demanda (lazy loading) e proteção de páginas baseada nos perfis de acesso definidos para cada usuário. A validação das permissões ocorre antes da renderização da página, garantindo que somente usuários autorizados possam acessar determinados módulos.

8.2.4 Gerenciamento de Estado Global

O gerenciamento de estado global foi realizado utilizando Redux Toolkit, permitindo o compartilhamento de informações entre diferentes partes da aplicação. Entre os estados controlados globalmente destacam-se: dados do usuário autenticado, informações da organização, configurações do sistema e dados temporários de formulários e filtros.

8.2.5 Resultados Obtidos na Interface

Ao final do desenvolvimento, foram implementadas interfaces para os principais módulos do sistema NURA, contemplando funcionalidades acadêmicas, administrativas e financeiras. Entre as principais telas desenvolvidas destacam-se: tela de autenticação, dashboard institucional, gestão de usuários, gestão de cursos e turmas, matrículas, controle financeiro, biblioteca, gestão de TCC e relatórios gerenciais.

8.3 Design System NURA

Com o crescimento da plataforma NURA e a expansão contínua dos módulos acadêmicos, administrativos e financeiros, tornou-se necessário estabelecer um conjunto padronizado de componentes e diretrizes visuais. O Design System centraliza definições de identidade visual, componentes reutilizáveis, padrões de interação e mecanismos de responsividade.

8.3.1 Motivação

Em aplicações de grande porte, a ausência de um sistema de design pode resultar em inconsistências visuais, duplicação de componentes e aumento dos custos de manutenção. O Design System do NURA foi criado para: padronizar cores, tipografia, escapamentos e estilos visuais; centralizar componentes reutilizáveis; garantir consistência entre todos os módulos da aplicação; facilitar a manutenção e evolução das interfaces; oferecer suporte nativo aos temas

claro e escuro; e aumentar a produtividade durante o desenvolvimento de novas funcionalidades.

8.3.2 Tokens de Design

Os elementos fundamentais da identidade visual foram definidos por meio de Design Tokens, armazenados como variáveis CSS globais. Esses tokens representam valores reutilizáveis utilizados em toda a interface, incluindo cores, superfícies, bordas e estados visuais. A utilização de Design Tokens facilita futuras alterações na identidade visual da plataforma, uma vez que modificações realizadas em um único local refletem automaticamente em todos os componentes dependentes.

8.3.3 Sistema de Dark Mode

O sistema NURA oferece suporte nativo aos modos claro (Light Mode) e escuro (Dark Mode), permitindo que o usuário escolha a aparência mais adequada às suas preferências e condições de uso. A alternância entre os temas é realizada por meio de um provedor responsável pelo gerenciamento do estado da aplicação e pela aplicação dinâmica das classes CSS correspondentes. Todos os componentes do sistema foram desenvolvidos considerando simultaneamente os dois temas, garantindo acessibilidade visual e consistência de comportamento.

8.3.4 Componentes do Design System

O Design System foi estruturado em três categorias principais: componentes primitivos, componentes compostos e componentes de layout.

Componentes Primitivos

Componente	Principais Variantes
Botao	Primário, Secundário, Fantasma, Destrutivo, Contorno e Link
Input	Pequeno, Médio e Grande
Select	Seleção com suporte a pesquisa
Caixa de Selecao	Checkbox estilizado
Interruptor	Alternador de estado (toggle)
Avatar	Exibição de imagem ou iniciais
Chip / Distintivo	Etiquetas e indicadores visuais

Componentes Compostos

Componente	Finalidade
Gaveta	Formulários laterais
Modal	Confirmações e interações pontuais
TabelaData	Exibição tabular com paginação
DialogoConfirmacao	Confirmações de ações críticas
FormField	Estruturação de campos de formulário

8.3.5 Impacto e Resultados

A adoção do Design System proporcionou benefícios significativos durante o desenvolvimento do sistema NURA. Entre os principais resultados observados destacam-se: padronização visual em mais de quarenta telas desenvolvidas; redução do tempo necessário para implementação de novas interfaces; maior reutilização de componentes entre módulos distintos; facilidade de manutenção e evolução da aplicação; uniformidade na implementação do tema escuro; e melhoria da experiência do usuário por meio de padrões consistentes de interação.

9 TESTES

9.1 Estratégia de Testes

O NURA adota a Pirâmide de Testes, priorizando testes unitários na base e testes E2E no topo, buscando equilíbrio entre velocidade de execução e confiança nas entregas.

9.2 Testes Unitários (Jest)

Os testes unitários cobrem a lógica de negócio dos services do backend, validando o comportamento isolado de cada componente do sistema.

9.3 Testes de Integração (Supertest)

Os testes de integração validam os endpoints da API REST em um banco de dados de teste isolado, garantindo que os módulos funcionem corretamente quando integrados.

9.4 Testes End-to-End (Cypress)

Os testes End-to-End validam os fluxos completos do usuário no frontend. Os fluxos cobertos incluem: login com credenciais validas, redirecionamento por papel (ADMIN para dashboard admin), logout e limpeza de sessão, criação de novo aluno e lançamento de notas.

9.5 Resultados dos Testes

Tipo	Total	Passando	Falhando	Cobertura
Unitários	[preencher]	[preencher]	[preencher]	[preencher]%
Integração	[preencher]	[preencher]	[preencher]	-
E2E (Cypress)	[preencher]	[preencher]	[preencher]	-

10 RESULTADOS ALCANCADOS

10.1 Sistema Desenvolvido

Ao término do período de desenvolvimento documentado neste trabalho, o NURA conta com os seguintes módulos implementados:

Modulo	Status	Funcionalidades
Autenticação	Completo	Login JWT, RBAC, multi-tenant
Gestão de Organizações	Completo	CRUD, ativação/desativação de módulos
Acadêmico	Parcial	Cursos, turmas, disciplinas, matrículas
Pessoas	Parcial	Cadastro de alunos, professores
Financeiro	Parcial	Mensalidades, pagamentos
Secretaria	Parcial	Requerimentos, documentos
Comunicação	Estruturado	-
TCC	Estruturado	-

10.2 Métricas do Projeto

Métrica	Valor
Módulos backend (NestJS)	30+
Domínios frontend (React)	40+
Papeis de usuário (RBAC)	15
Telas implementadas (frontend)	50+
Endpoints REST documentados	[preencher]
Linhas de código (TypeScript)	[preencher]
Cobertura de testes (backend)	[preencher]%

Métricas de Qualidade

Métrica	Valor
Erros TypeScript (tsc --noEmit)	0
Warnings ESLint	0
Ocorrências de: any no código	0
Cobertura de testes (unitários)	[preencher]%

Testes E2E passando	[preencher]
---------------------	-------------

Métricas de Infraestrutura

Métrica	Valor
Tempo de build (TypeScript)	~30s
Tempo de deploy (GitHub Actions)	~1min30s
Uptime do servidor dev	99%+
Versão Node.js	18+ LTS
Versão PostgreSQL	15+

10.3 Interface do Sistema

As capturas de tela das principais interfaces desenvolvidas encontram-se apresentadas a seguir. As telas documentadas incluem: tela de login, dashboard do administrador, gestão de organizações, listagem de alunos, modulo financeiro e painel do aluno.

10.4 Validação em Ambiente Real

O sistema foi implantado em servidor de produção e está sendo utilizado por organizações reais para fins de demonstração e validação.

10.5 Objetivos Alcançados

Objetivo Específico	Alcançado?	Observações
Arquitetura multi-tenant	Sim	Row-level isolation implementado
RBAC com 15 papéis	Sim	Todos os papéis funcionais
Interface React responsiva	Sim	Light/dark mode, responsiva
Design system centralizado	Sim	Tokens em index.css
Modulos integrados	Parcial	7 de ~20 modulos completos
Testes automatizados	Parcial	Unitarios e E2E implementados

Evolução do Projeto (Timeline)

Periodo	Marco Principal
Abr 2026 (semana 1)	Fundacao: monorepo, auth, design system

Abr 2026 (semana 2)	Gestao de organizacoes + RBAC completo
Abr 2026 (semana 3)	Auditoria: 40+ telas com dark mode corrigido
Abr 2026 (semana 4)	Sidebar completa + 15 papeis integrados
Mai 2026	Testes E2E, documentação e TCC

11 CRONOGRAMA

11.1 Cronograma Planejado

Atividade	Mar	Abr	Mai	Jun	Jul
Definição do tema e escopo	X				
Pesquisa bibliográfica	X	X			
Configuração da infraestrutura		X			
Arquitetura e design system		X			
Modulo de autenticação e RBAC		X			
Módulos acadêmico e pessoas		X	X		
Modulo financeiro			X		
Módulos de secretaria e comunicação			X		
Testes automatizados (E2E)			X	X	
Redação do TCC			X	X	
Revisão e ajustes finais				X	
Entrega do TCC					X
Apresentação a banca					X

11.2 Cronograma Realizado

Atividade	Status	Data de Conclusao
Configuração do monorepo	Concluído	Abr 2026
Design system e tokens	Concluído	Abr 2026
Autenticacao JWT + RBAC	Concluído	Abr 2026
Gestao multi-tenant	Concluído	Abr 2026
Sidebar e navegação	Concluído	Abr/Mai 2026
Modulo acadêmico (parcial)	Em andamento	-
Testes E2E (Cypress)	Em andamento	-
Redacao do TCC	Em andamento	-

12 CONCLUSÃO

12.1 Considerações Finais

O presente trabalho apresentou o desenvolvimento do NURA, uma plataforma SaaS multi-tenant de gestão educacional construída com tecnologias modernas de código aberto. O problema de pesquisa proposto, sobre como centralizar e automatizar os processos de gestão acadêmica de múltiplas instituições de ensino superior em uma única plataforma com isolamento seguro de dados entre organizações, foi respondido pela implementação de uma arquitetura multi-tenant com isolamento por linha de banco de dados, mediado pelo TenantMiddleware e validado por um sistema de controle de acesso baseado em papéis (RBAC) com 15 perfis distintos.

Os objetivos específicos foram atingidos de forma satisfatória. A arquitetura multi-tenant foi projetada e implementada com NestJS e Prisma ORM, garantindo que cada organização opere em um contexto completamente isolado das demais. O sistema RBAC com 15 papéis cobre todos os atores institucionais relevantes, desde o administrador da plataforma até candidatos ao processo seletivo e egressos. O frontend, desenvolvido em React 18 com TypeScript e Tailwind CSS v4, apresenta interfaces responsivas com suporte nativo a modo claro e escuro, unificadas por um design system centralizado que garante consistência visual em mais de 40 telas da plataforma.

A validação do sistema em ambiente de produção real, com organizações e usuários reais acessando os módulos desenvolvidos, constitui o principal diferencial deste trabalho em relação a projetos acadêmicos de caráter estritamente prototípico. A presença do NURA em produção válida não apenas a correção funcional do sistema, mas sua adequação ao contexto real de uso por instituições de ensino.

Do ponto de vista da Engenharia de Software, este trabalho demonstrou a aplicabilidade de padrões arquiteturais modernos, como DDD, RBAC, multi-tenancy e design system, em um projeto de escala real, conduzido por uma equipe enxuta mas com processos rigorosos de qualidade de código: TypeScript estrito, ESLint, Husky, testes automatizados e CI/CD com GitHub Actions.

12.2 Limitações

O trabalho apresenta limitações que devem ser reconhecidas para contextualização adequada dos resultados.

Completude dos módulos: Dos mais de 20 módulos planejados na arquitetura do NURA, aproximadamente 7 encontram-se completamente implementados no período abrangido por este trabalho. Os demais módulos possuem estrutura criada e backend implementado, mas aguardam a conclusão do frontend correspondente.

Cobertura de testes: Os testes unitários do backend cobrem principalmente os módulos de autenticação e gestão de organizações, que foram priorizados por sua criticidade. A expansão da cobertura de testes para todos os módulos é um processo em andamento.

Validação com usuários finais: Embora o sistema esteja em produção, a validação formal com usuários finais seguindo protocolos de usabilidade, testes A/B, avaliação heurística e System Usability Scale (SUS), não foi conduzida dentro do escopo deste trabalho.

Conformidade LGPD: As boas práticas de segurança implementadas alinham o sistema com os princípios da LGPD. Contudo, uma análise formal de conformidade, com mapeamento completo do fluxo de dados pessoais e relatório de impacto à proteção de dados (RIPD), não foi conduzida.

12.3 Trabalhos Futuros

A natureza modular e extensível do NURA abre amplas oportunidades para continuidade. Os seguintes trabalhos futuros são sugeridos:

Completude dos módulos restantes: A finalização dos módulos de Estágio, Biblioteca Digital, Extensão, Pesquisa Científica, Monitoria e Portaria, aproveitando a infraestrutura backend já criada, e a continuidade natural mais imediata.

Aplicativo mobile nativo: O desenvolvimento de aplicativos iOS e Android com React Native, consumindo a mesma API REST já disponível, expandiria significativamente a acessibilidade da plataforma.

Assistente de Inteligência Artificial (Lyra): A integração de um assistente de IA para automação de respostas a requerimentos, análise preditiva de risco de evasão e geração de relatórios acadêmicos representa uma frente estratégica de inovação.

Integrações com sistemas externos: A integração com o INEP para exportação de dados do Censo Educacional Superior, com bancos para conciliação automática de pagamentos e com prefeituras para emissão de NFSe ampliariam o valor da plataforma.

Avaliação formal de usabilidade: A condução de estudos de usabilidade com usuários reais, utilizando métodos como avaliação heurística e SUS, permitiria identificar oportunidades de melhoria na interface.

Testes de carga e escalabilidade: A realização de stress tests com ferramentas como k6 ou JMeter para validar o comportamento do sistema sob carga de dezenas de organizações e milhares de usuários simultâneos é fundamental para o crescimento sustentável da plataforma.

REFERENCIAS

ABSTARTUPS. Mapeamento do Ecossistema Brasileiro de Startups. Sao Paulo: Abstartups, 2023. Disponivel em: <https://abstartups.com.br>. Acesso em: 10 mai. 2026.

ASSOCIACAO BRASILEIRA DE NORMAS TECNICAS. NBR 6023: informacao e documentacao: referencias - elaboracao. Rio de Janeiro: ABNT, 2018.

BECK, Kent et al. Manifesto para o Desenvolvimento Agil de Software. 2001. Disponivel em: <https://agilemanifesto.org>. Acesso em: 10 mai. 2026.

BEZEMER, Cor-Paul; ZAIDMAN, Andy. Multi-tenant SaaS applications: maintenance dream or nightmare? In: INTERNATIONAL WORKSHOP ON PRINCIPLES OF SOFTWARE EVOLUTION AND SOFTWARE EVOLUTION, 2010, Antwerp. Proceedings... ACM, 2010. p. 88-92.

EVANS, Eric. Domain-Driven Design: Tackling Complexity in the Heart of Software. Boston: Addison-Wesley, 2003.

FOWLER, Martin. Patterns of Enterprise Application Architecture. Boston: Addison-Wesley, 2002.

HEJLSBERG, Anders et al. TypeScript Handbook. Microsoft, 2023. Disponivel em: <https://www.typescriptlang.org/docs/handbook/>. Acesso em: 10 mai. 2026.

INSTITUTO NACIONAL DE ESTUDOS E PESQUISAS EDUCACIONAIS ANISIO TEIXEIRA. Censo da Educacao Superior 2023: notas estatisticas. Brasilia: INEP/MEC, 2023. Disponivel em: <https://www.gov.br/inep>. Acesso em: 10 mai. 2026.

LOBO, Roberto; LOBO, Maria Beatriz. O Futuro da Educacao Superior no Brasil. Sao Paulo: Instituto Lobo, 2022.

MELL, Peter; GRANCE, Timothy. The NIST Definition of Cloud Computing. NIST Special Publication 800-145. Gaithersburg: National Institute of Standards and Technology, 2011.

NAKAYAMA, Marina Keiko; PILLA, Bianca Santana; RISSI, Mauricio. A Importancia dos Sistemas de Informacao na Gestao Educacional. Florianopolis: UFSC, 2016.

NESTJS. NestJS Documentation. 2024. Disponivel em: <https://docs.nestjs.com>. Acesso em: 10 mai. 2026.

NODE.JS FOUNDATION. Node.js Documentation. 2024. Disponivel em: <https://nodejs.org/docs>. Acesso em: 10 mai. 2026.

POSTGRES SQL GLOBAL DEVELOPMENT GROUP. PostgreSQL 15 Documentation. 2024. Disponível em: <https://www.postgresql.org/docs/15/>. Acesso em: 10 mai. 2026.

PRESSMAN, Roger S.; MAXIM, Bruce R. Engenharia de Software: Uma Abordagem Profissional. 8. ed. Porto Alegre: AMGH, 2016.

PRISMA. Prisma Documentation. 2024. Disponível em: <https://www.prisma.io/docs>. Acesso em: 10 mai. 2026.

REACT. React Documentation. Meta Platforms, 2024. Disponível em: <https://react.dev>. Acesso em: 10 mai. 2026.

SANDHU, Ravi S. et al. Role-based access control models. IEEE Computer, v. 29, n. 2, p. 38-47, fev. 1996.

SOMMERVILLE, Ian. Engenharia de Software. 10. ed. Sao Paulo: Pearson Education do Brasil, 2018.

TAILWIND CSS. Tailwind CSS Documentation. 2024. Disponível em: <https://tailwindcss.com/docs>. Acesso em: 10 mai. 2026.

TILKOV, Stefan; VINOSKI, Steve. Node.js: Using JavaScript to Build High-Performance Network Programs. IEEE Internet Computing, v. 14, n. 6, p. 80-83, nov./dez. 2010.

WEISSMAN, Craig; BOBROWSKI, Steve. The design of the force.com multitenant internet application development platform. In: ACM SIGMOD INTERNATIONAL CONFERENCE ON MANAGEMENT OF DATA, 2009, Providence. Proceedings... ACM, 2009. p. 889-896.

YIN, Robert K. Estudo de Caso: Planejamento e Metodos. 5. ed. Porto Alegre: Bookman, 2015.